# Group 29 Progress Report: Early Prediction of Student Dropout and Academic Success

**Anam Khan and Haniya Kashif**
{khana489,kashih2}@mcmaster.ca

## 1 Introduction

Universities usually learn that a student is struggling only after the student has withdrawn or failed enough courses that recovery is unrealistic. Dropout is costly for students (debt without a degree), for institutions (lost tuition and weaker graduation statistics) and for society (a lower return on public education spending). Advisors cannot manually monitor thousands of students, so an automated early-warning system could let staff offer tutoring, financial aid or counselling while it still matters.

We build such a system as a three-class classifier (Dropout, Enrolled, Graduate) on tabular student records. Our central question is *how early* a student's outcome can be predicted reliably. We ask the same question at three points in time with progressively more information: at enrollment (Stage 1), after semester 1 (Stage 2) and after semester 2 (Stage 3). A moderately accurate Stage 1 model may be more useful than a more accurate Stage 3 model, because only an early prediction leaves time to help. Anticipated challenges are class imbalance, the ambiguity of the "Enrolled" label, weak early signal, feature leakage across stages, and fairness.

This report describes our preliminary implementation: the dataset and preprocessing, leakage-safe stage-wise feature sets, a majority-class baseline, and three learned models (logistic regression, random forest and gradient boosting) evaluated at all three stages. Even at this early point, all learned models clearly beat the majority baseline (0.22 macro F1). Test macro F1 rises from about 0.52 at Stage 1 to about 0.71 at Stage 3, with most of the gain arriving once semester-1 information is available.

## 2 Related Work

**Student retention and dropout prediction** Student retention has long been studied in education research; Tinto (1975) gave an influential theoretical account of dropout from higher education based on academic and social integration. More recent work applies machine learning to institutional records. Our dataset was created at a Portuguese polytechnic institute to support early identification of students at risk and was introduced by Martins et al. (2021), who used only information known at enrollment, tested synthetic oversampling to handle class imbalance, and compared standard classifiers with boosting algorithms, finding that boosting worked better; they listed first-year academic performance as future work. The same group later studied prediction at several phases of the enrollment year and found that random forests, combined with imbalance-handling strategies, performed best (Martins et al., 2023). Realinho et al. (2022) document the dataset itself: it merges several disjoint institutional databases and labels each student as Dropout, Enrolled or Graduate at the end of the normal course duration. Our work is closest to these phased studies, but differs in assigning each variable to a stage conservatively to avoid leakage (for example, placing payment status after enrollment), excluding Nationality from the main feature set, and planning explicit fairness, proxy-feature and calibration analyses across stages. These papers motivate the task and provide the data. Our project differs in that we explicitly build one feature set per prediction point, assigning variables conservatively to avoid leakage, and compare models across those stages rather than treating all features as available at once.

**Models for tabular data** For tabular data with mixed numeric and categorical features, tree ensembles are strong default choices. Random forests (Breiman, 2001) average many decorrelated trees,

and gradient boosting (Friedman, 2001) builds trees sequentially to correct earlier errors. Efficient implementations such as XGBoost (Chen and Guestrin, 2016) and LightGBM (Ke et al., 2017) are widely used; the histogram-based boosting in scikit-learn that we use follows the LightGBM design. Grinsztajn et al. (2022) found that tree-based models often remain ahead of deep learning on medium-sized tabular datasets, which is why we focus on tree ensembles and use logistic regression as an interpretable baseline.

**Imbalance, calibration and fairness** Because our classes are imbalanced, we also draw on work on imbalance and probability quality. Chawla et al. (2002) proposed SMOTE oversampling, which we plan to compare against class weighting. Niculescu-Mizil and Caruana (2005) showed that predicted probabilities from many classifiers are poorly calibrated, motivating our planned calibration checks. For fairness, Hardt et al. (2016) define group-level error criteria such as equality of opportunity, which we will use to compare error rates across groups (also why we exclude using nationality for now), and Lundberg and Lee (2017) provide SHAP, one option for interpreting predictions.

## 3 Dataset

We use the public UCI "Predict Students' Dropout and Academic Success" dataset (CC BY 4.0), which contains 4,424 student records with 36 features and one target label (Dropout, Enrolled or Graduate) drawn from the institution's records, so no manual annotation was needed. Features cover demographics, application and enrollment information, previous education, parents' education and occupation, scholarship and payment status, first- and second-semester curricular unit statistics, and three macroeconomic indicators (unemployment, inflation, GDP). The data are included with our submission (`data.csv`).

**Class distribution** The classes are imbalanced: Graduate is 49.9% of students (2,209), Dropout 32.1% (1,421) and Enrolled 17.9% (794) (Figure 1). A majority-class predictor therefore reaches about 50% accuracy but only 0.222 macro F1.

**Preprocessing** The dataset documentation reports that anomalies and missing values were handled by its creators; we verified that there are no missing values and checked for duplicate rows. We stripped stray whitespace from column names,

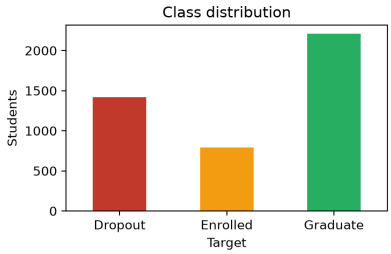



Figure 1: Class distribution of the 4,424 students.

mapped the target to integers (Dropout=0, Enrolled=1, Graduate=2), and dropped the Nationality column from the main feature set because of fairness concerns. Eight nominal code columns (marital status, application mode, course, previous qualification, and the mothers' and fathers' qualification and occupation) are treated as categorical, since their integer codes carry no numeric meaning. All other columns are used as numeric or binary values.

**Split** We make one stratified 80/20 split (3,539 training and 885 test students) with a fixed random seed. The same split is reused for every stage and model so that stage comparisons are fair. The test set is used only to report final numbers.

## 4 Features

**Stage-wise feature sets** We assign each variable to the earliest stage at which it would be known (Table 1). Stage 1 contains demographic, application, prior-education, parental, scholarship and macroeconomic variables. Stage 2 adds first-semester curricular unit statistics (credited, enrolled, evaluated, approved, grade, without evaluations) and the payment variables Tuition fees up to date and Debtor, which we place in Stage 2 because payment status can change after enrollment and may already reflect a student's decision to leave. Stage 3 adds the second-semester statistics. Scholarship status stays in Stage 1 because scholarships are generally awarded as part of enrollment, and macroeconomic indicators stay in Stage 1 because they do not depend on the student. Each model only receives its own stage's features, which prevents leakage from later semesters into earlier predictions.

**Representation** For logistic regression we one-hot encode the eight nominal columns and standardize the numeric columns. For tree models we ordinal-encode the nominal columns and leave numeric columns unscaled, since trees are insensitive

| Stage | Adds | # features |
|---|---|---|
| 1 | enrollment, background, macro | 21 |
| 2 | payment status, semester 1 | 29 |
| 3 | semester 2 | 35 |

Table 1: Feature sets per stage (Nationality excluded).

to monotone rescaling. We have not yet done any hand-crafted feature engineering or automatic feature selection; the only selection is the stage grouping and the removal of Nationality. Ratio features (for example units approved divided by units enrolled) and a semester-1 to semester-2 grade change are candidates for future work.

## 5 Implementation

All models are implemented with scikit-learn. Each model is a pipeline of preprocessing followed by a classifier, so preprocessing (encoding and scaling) is fit only on the training portion of each cross-validation fold and never sees validation or test students.

**Baseline and models** The required baseline is a majority-class predictor, which always predicts Graduate (49.9% of students). We train three learned models: (1) multinomial logistic regression, an interpretable linear baseline; (2) a random forest; and (3) histogram-based gradient boosting, the model we expect to be strongest on tabular data. Logistic regression, random forests and boosting are standard choices for this kind of data and give us a progression from a linear model to bagged trees to boosted trees. Each is trained separately at each of the three stages, giving nine learned models plus the baseline.

**Class imbalance** All three learned models use balanced class weights, $w_c = N/(KN_c)$ with $K = 3$ classes and $N_c$ training students in class $c$, so that errors on the smaller Dropout and Enrolled classes are penalized more heavily than errors on Graduate.

**Loss functions and optimization** *Logistic regression* minimizes the L2-regularized, class-weighted multinomial cross-entropy (in scikit-learn's parameterization)

$$\min_{\theta} \ \tfrac{1}{2}\|\theta\|_2^2 \ + \ C \sum_{i=1}^{N} w_{y_i} \left(-\log p_{i,y_i}\right),$$

where $p_{i,y_i}$ is the softmax probability assigned to student $i$'s true class and the intercept is not penalized. A larger $C$ means weaker regularization. It is optimized with L-BFGS, a quasi-Newton optimization method. It's the default solver in scikit-learn's LogisticRegression. It's a good fit for logistic regression because the loss is smooth and convex, so it converges quickly and reliably. *Gradient boosting* minimizes the same multiclass cross-entropy, adding one small regression tree per iteration that is fit to the gradient of the loss with respect to the current predictions, scaled by a learning rate. The histogram-based implementation first bins each feature, which makes split search fast. A 10% internal validation split stops training early once the validation loss stops improving. *Random forest* has no single global loss: each tree is trained on a bootstrap sample, considers a random subset of features at every split, and chooses splits by Gini impurity; the forest's predicted class probabilities are averaged over trees.

**Hyperparameter tuning** For each stage and model we run randomized search with 5-fold stratified cross-validation on the training set, scoring macro F1, and refit the best configuration on the full training set. We sample 15 configurations for logistic regression and 20 each for the random forest and gradient boosting. The search spaces are: regularization strength $C$ (log-uniform in $[10^{-3}, 10^{2}]$) for logistic regression; number of trees, maximum depth, minimum leaf size and `max_features` for the random forest; and learning rate, depth, number of leaves, minimum leaf size, L2 penalty and number of iterations for boosting. Selected values are saved in `best_params.json`. We tune on macro F1 rather than on dropout recall alone so that the Enrolled class is not sacrificed entirely.

**Ablations** So far, the stage comparison is itself our main ablation: it measures what each additional group of features (semester 1, then semester 2) contributes to every model. We have not yet ablated imbalance handling or the macroeconomic features, and these are planned for the next phase.

**Implementation issues and limitations** Firstly, the tree models receive the nominal code columns as ordinal-encoded integers, which imposes an arbitrary order on categories such as course; trees can still isolate individual codes by repeated splits, but native categorical handling or target encoding may do better and is something we may test. Second, because hyperparameters are tuned on the same folds used to produce the validation scores, those scores are slightly optimistic, so we rely on the

held-out test set for final comparisons. Third, all results come from a single train/test split, so differences of a couple of points in macro F1 between models are within the noise of an 885-student test set.

## 6 Results and Evaluation

**Evaluation strategy** We use the single stratified train/test split from Section 3. Validation estimates come from 5-fold stratified cross-validated out-of-fold predictions on the training set; test estimates come from the refit model on the held-out test set. Macro F1 and dropout recall are our main metrics, because accuracy is misleading with imbalanced classes and a missed dropout is the costliest error. We also report dropout precision, Enrolled recall, balanced accuracy, log loss and the multiclass Brier score, and examine confusion matrices.

**Findings** Table 2 summarizes results. (i) All learned models far exceed the majority baseline (0.222 macro F1, zero dropout recall). (ii) Waiting helps, mostly once. The best test macro F1 rises from about 0.52 at Stage 1 to about 0.68 at Stage 2 and 0.71 at Stage 3, so most of the gain arrives with semester-1 information, while semester 2 adds comparatively little. (iii) At every stage the three learned models are within about 0.02 test macro F1 of each other, which on 885 test students is too close to declare a winner; validation scores are similarly close, with gradient boosting marginally ahead. Logistic regression is surprisingly competitive, suggesting much of the signal is close to linear in the semester performance features. (iv) Enrolled is the hardest class (recall 0.35–0.47 at Stage 1, 0.56–0.69 at Stage 3), consistent with its unresolved outcome. (v) Validation and test macro F1 differ by at most about 0.04, so we are not seeing severe overfitting to the tuning folds, although validation scores are slightly optimistic because hyperparameters were chosen on the same folds. (vi) Test log loss improves from about 0.94–0.97 at Stage 1 to about 0.60–0.63 at Stage 3.

## 7 Feedback and Plans

### TA feedback

**Reflection** Our results show that Stage 1 is weak (macro F1 about 0.5), that the learned models are nearly tied, and that Enrolled is the main source of error. Because the models are so close, we cannot yet claim that boosting is better than a regularized linear model, and our single split on 885 test students gives noisy comparisons.

**Plans** (1) Reduce noise in comparisons by repeating cross-validation with several seeds and reporting confidence intervals, and test whether model differences are significant. (2) Improve Enrolled and dropout recall with threshold tuning on predicted probabilities and compare class weighting against SMOTE (Chawla et al., 2002). (3) Add engineered features such as approval ratios and semester-to-semester changes, and ablate the macroeconomic features. (4) Check calibration of predicted probabilities and calibrate if needed. (5) Interpret the best models with permutation importance and SHAP. (6) Run fairness checks (for example by gender and scholarship status, using group-wise recall) and test whether features such as International status, Displaced or parental variables act as proxies for nationality; if time permits, run the secondary experiment including Nationality. (7) If the three-class task proves too overlapping for the Enrolled class, discuss it as a limitation rather than switching to a binary task, following the TA's advice.

## Team Contributions

We both worked on and finished the code together. We both checked the final results table against the pipeline output. **Anam Khan** wrote the Dataset, Features, Results and Evaluation, and Feedback and Plans sections. **Haniya Kashif** wrote the Introduction, Related Work and Implementation sections, reviewed the code and comments, and compiled the team contributions.

## References


Leo Breiman. 2001. Random forests. *Machine Learning*, 45(1):5–32.

Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. 2002. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357.

Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794.

Jerome H. Friedman. 2001. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232.

| Stage | Model | Val F1 | Test F1 | Bal. Acc. | Dropout R | Dropout P | Enrolled R |
|---|---|---|---|---|---|---|---|
| — | Majority | .222 | .222 | .333 | .000 | .000 | .000 |
| 1 | LogReg | .533 | .520 | .531 | .563 | .604 | .465 |
| 1 | RandomForest | .537 | .494 | .499 | .553 | .520 | .346 |
| 1 | GradBoost | .538 | .510 | .519 | .542 | .572 | .434 |
| 2 | LogReg | .682 | .664 | .673 | .658 | .803 | .585 |
| 2 | RandomForest | .691 | .684 | .686 | .701 | .787 | .528 |
| 2 | GradBoost | .696 | .679 | .691 | .669 | .812 | .629 |
| 3 | LogReg | .716 | .701 | .715 | .683 | .822 | .667 |
| 3 | RandomForest | .721 | .715 | .715 | .718 | .826 | .560 |
| 3 | GradBoost | .724 | .710 | .726 | .704 | .840 | .692 |

Table 2: Macro F1 (validation = cross-validated, and test), and test balanced accuracy, dropout recall (R), dropout precision (P) and Enrolled recall. The majority baseline always predicts Graduate and is identical at all stages.

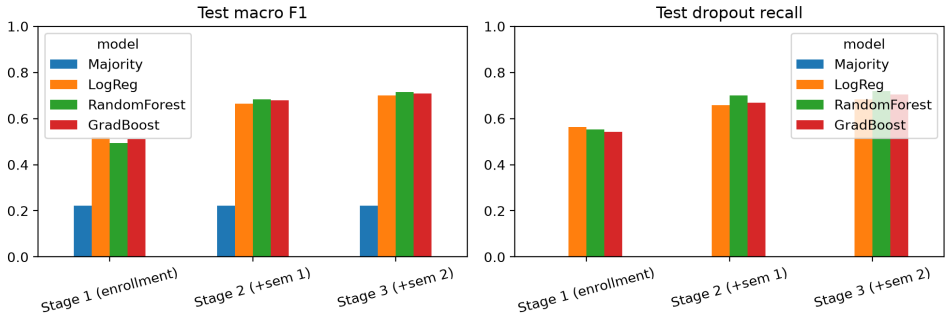



Figure 2: Test macro F1 and dropout recall by stage and model.


Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. 2022. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 35.

Moritz Hardt, Eric Price, and Nathan Srebro. 2016. Equality of opportunity in supervised learning. In *Advances in Neural Information Processing Systems*, volume 29, pages 3315–3323.

Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, pages 3146–3154.

Scott M. Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30, pages 4765–4774.

Mónica V. Martins, Daniel Tolledo, Jorge Machado, Luís M. T. Baptista, and Valentim Realinho. 2021. Early prediction of student's performance in higher education: A case study. In *Trends and Applications in Information Systems and Technologies (WorldCIST 2021), Volume 1*, Advances in Intelligent Systems and Computing, pages 166–175. Springer. DOI: 10.1007/978-3-030-72657-7_16.

Mónica V. Martins, Luís Baptista, Jorge Machado, and Valentim Realinho. 2023. Multi-class phased prediction of academic performance and dropout in higher education. *Applied Sciences*, 13(8):4702.

Alexandru Niculescu-Mizil and Rich Caruana. 2005. Predicting good probabilities with supervised learning. In *Proceedings of the 22nd International Conference on Machine Learning*, pages 625–632.

Valentim Realinho, Jorge Machado, Luís Baptista, and Mónica V. Martins. 2022. Predicting student dropout and academic success. *Data*, 7(11):146.

Vincent Tinto. 1975. Dropout from higher education: A theoretical synthesis of recent research. *Review of Educational Research*, 45(1):89–125.

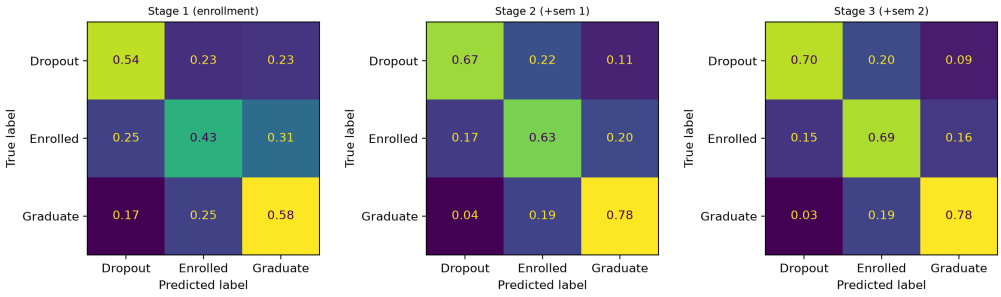



Figure 3: Row-normalized test confusion matrices for gradient boosting at each stage.